TECH THINK²

Inteligência Artificial
além da nuvem

IA Generativa no Desenvolvimento

github.com/tech2think/python-learning

M4 A Evolução da IA no Desenvolvimento

2022

A ERA DO CHAT WEB (CHATGPT)

Interação isolada no navegador. O desenvolvedor precisava explicar todo o contexto, copiar o código gerado, colar na IDE, testar e voltar ao navegador se houvesse erro.

2023

A ERA DOS COPILOTS (EXTENSÕES)

A IA entra na IDE. O foco muda para o **autocompletar inteligente** e chats embutidos que já leem o arquivo aberto. Menos "copiar/colar", mas ainda muito reativo.

Hoje

A ERA DOS AGENTES (CLAUDE CODE / GEMINI CLI)

A IA opera no terminal. Ela não apenas sugere, mas **executa**. Lê a estrutura do projeto, roda comandos, executa testes, encontra erros e propõe soluções de forma autônoma (Agentes).

O que é Claude Code?

Claude Code é uma ferramenta de linha de comando da Anthropic que integra o modelo Claude diretamente ao terminal e ao seu editor de código.

Ele lê e escreve arquivos, executa comandos, entende o contexto do seu projeto e **colabora como um desenvolvedor experiente** — mas você mantém o controle total.

Diferente de autocompletar de IDE: Claude Code **entende o projeto inteiro** e pode refatorar, documentar e gerar testes de forma contextual.

```
bash — claude code

$ claude "refactor this data pipeline"

[Claude Thinking...]

I'll analyze the project structure...

✓ Found 3 related files in /src
✓ Running tests... (12/12 passed)

I've prepared a plan to optimize the Snowflake
connection. Shall I proceed? (y/n)

[PROPOSED CHANGES]
+ import snowflake.connector
- old_connection_method()
+ new_optimized_pool()
```

CASOS DE USO PARA ANALISTAS

- Gerar queries SQL para Snowflake
- Criar funções de validação de dados de UC
- Documentar pipelines automaticamente
- Refatorar notebooks em scripts produtivos

M4 Pré-requisito: Instalando o Node.js

> Tanto o **Claude Code** quanto o **Gemini CLI** são ferramentas desenvolvidas em JavaScript/TypeScript e distribuídas como pacotes do Node.js (via `npm`).

Por que precisamos disso?

- O **Node.js** é o motor que executa o código dessas ferramentas no seu computador, fora do navegador.
- O **NPM** (Node Package Manager) é o instalador automático, que baixa a IA e todas as suas dependências.

PASSO A PASSO

1. Windows / macOS

Acesse **nodejs.org** e baixe a versão **LTS (Long Term Support)**. Siga o instalador padrão (Next, Next, Finish).

2. Linux (Debian/Ubuntu)

```
# Instale o NVM (Node Version Manager)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash

# Instale a versão mais recente LTS
nvm install --lts
```

3. Verificar Instalação

```
node -v # Exemplo: v20.11.0
npm -v # Exemplo: 10.2.4
```

M4 Instalação e Configuração

Pré-requisitos

- Node.js 18+ instalado
- Conta na Anthropic (anthropic.com)
- Chave de API (API Key)

Instalação

```
# Instalar globalmente via npm
npm install -g @anthropic-ai/claude-code

# Configurar a API Key
export ANTHROPIC_API_KEY="sk-ant-..."

# Ou adicionar ao ~/.bashrc / ~/.zshrc
echo 'export ANTHROPIC_API_KEY="sk-ant-..."' \
  >> ~/.bashrc
```

Primeiros comandos

```
# Iniciar Claude Code no projeto
cd /meu/projeto
claude

# Modo não-interativo (comando único)
claude -p "Explique este arquivo"

# Verificar versão
claude --version
```

> Ao iniciar o Claude Code em um projeto com `CLAUDE.md`, ele carrega automaticamente o contexto do projeto.

M4 Estrutura de Projetos com CLAUDE.md

O que é o CLAUDE.md?

É um arquivo Markdown na raiz do projeto que fornece contexto persistente ao Claude Code: quem você é, o que o projeto faz, quais padrões seguir e o que **não** fazer.

> O CLAUDE.md é lido a **cada sessão**. Mantenha-o atualizado — é a memória do seu projeto.

Seções recomendadas

- Contexto do negócio e domínio
- Stack tecnológica e versões
- Convenções de nomenclatura
- Padrões de código e estilo
- O que Claude NUNCA deve fazer
- Comandos úteis do projeto

Exemplo — qualidade de dados

```
# Projeto: Qualidade Cadastro UC

## Contexto
Pipeline de qualidade do cadastro de
consumidores de uma distribuidora de
energia. Dados no Snowflake.

## Stack
- Python 3.11, pandas, great-expectations
- Snowflake (schema: CADASTRO)
- Tabela principal: CONSUMIDORES_UC

## Convenções
- Nomes em snake_case
- Prefixo vw_ para views
- Queries parametrizadas (sem f-strings
  em SQL para evitar SQL injection)

## Nunca
- Fazer DELETE/UPDATE sem WHERE
- Commitar credenciais no .env
```

M4 Skills e Prompts Customizados

O que são Skills?

Skills são arquivos Markdown que definem tarefas reutilizáveis para o Claude Code. Você invoca com `/nomeskill` e ele executa a tarefa descrita.

```
# .claude/commands/data-quality.md
---
description: Executa análise de qualidade
  no DataFrame `df`
---

Analise a qualidade dos dados no
DataFrame `df`:
1. % nulos por coluna
2. Duplicatas na chave primária
3. Outliers em colunas numéricas
4. Gere um relatório em Markdown
```

> Salve skills em `.claude/commands/` no projeto.

Prompt engineering para dados

✅ Prompt eficaz:

```
"Crie uma função Python que recebe um
DataFrame pandas com a tabela CONSUMIDORES_UC
(colunas: cod_consumidor, cpf_cnpj,
cod_modalidade_tarifaria, vlr_consumo_medio_kwh)
e retorna um dict com:
- pct_nulos: % nulos por coluna
- duplicatas: contagem de cod_consumidor
  duplicados
- outliers_consumo: registros com consumo
  acima de 3 desvios padrão
Use type hints e docstring em português."
```

❌ Prompt ineficaz:

```
"faça uma função de qualidade de dados"
```

M4 Markdown para Documentação Técnica

Por que Markdown?

- Legível como texto puro
- Renderizado em GitHub, Jupyter, VS Code
- Base para dicionários de dados
- Gerado automaticamente pelo Claude Code
- Versiona junto com o código (git)

> O Claude Code lê e **gera** Markdown. Use-o para criar e manter documentação técnica automaticamente.

Elementos essenciais

```
# Título Principal
## Seção
### Subseção

**negrito**, *itálico*, `código inline`

- Item de lista
- Outro item

| Coluna A | Coluna B |
|----------|----------|
| Valor 1 | Valor 2 |

```python
# bloco de código com sintaxe
def funcao():
  pass
```

> Citação ou nota importante
```

M5 Snowflake + Python: Arquitetura

Componentes Snowflake

Account	Instância da sua empresa
Warehouse	Cluster de computação (VXS a 6XL)
Database	Banco de dados lógico
Schema	Agrupamento de tabelas/views

> Snowflake cobra por **tempo de execução do warehouse**. Sempre suspenda ao terminar análises longas.

Conexão Python

```python
import snowflake.connector
from dotenv import load_dotenv
import os

load_dotenv()

conn = snowflake.connector.connect(
  account=os.getenv("SF_ACCOUNT"),
  user=os.getenv("SF_USER"),
  password=os.getenv("SF_PASSWORD"),
  warehouse=os.getenv("SF_WAREHOUSE"),
  database=os.getenv("SF_DATABASE"),
  schema=os.getenv("SF_SCHEMA"),
)

cursor = conn.cursor()
cursor.execute("SELECT CURRENT_VERSION()")
print(cursor.fetchone()[0])
```

M5 Projeto: Qualidade de Dados de Consumidores

Objetivo: Construir um pipeline Python que conecta ao Snowflake, analisa a qualidade do cadastro de UCs (Unidades Consumidoras), gera um relatório com score por registro e identifica os principais problemas para correção.

1. EXTRAÇÃO

- Conectar ao Snowflake
- Carregar tabela CONSUMIDORES_UC
- Filtros por data de competência

2. ANÁLISE DE QUALIDADE

- Completude por campo
- Validação CPF/CNPJ
- Consistência tarifária
- Outliers de consumo

3. RELATÓRIO

- Score por UC (0 a 1)
- Dashboard Plotly
- Exportar Excel com issues
- Envio de alertas

M5 Agente Gemini com Ferramentas Snowflake

1. Definir a ferramenta

```python
def analisar_qualidade_amostra(limite: int = 100) ->
dict:
  """Analisa completude dos campos obrigatórios
  em uma amostra de consumidores."""
  query = f"""
    SELECT COUNT(*) AS TOTAL,
    SUM(CASE WHEN MEDIDOR_ID IS NULL
        THEN 1 ELSE 0 END) AS SEM_MEDIDOR
    FROM TAB_CADASTRO
    LIMIT {limite}
  """
  df = read_sql_to_dataframe(query)
  return df.iloc[0].to_dict()
```

2. Loop do agente

```python
response = client.models.generate_content(
  model="gemini-2.0-flash",
  contents=messages,
  config=types.GenerateContentConfig(
    tools=TOOLS,
    temperature=0.1,
  ),
)

# Gemini decide chamar a ferramenta
fc = response.candidates[0].content.parts[0]
resultado = TOOL_MAP[fc.function_call.name](
  **dict(fc.function_call.args)
)
```

FLUXO DE EXECUÇÃO

Pergunta → Gemini decide tool → **Snowflake executa** → Resultado ao Gemini → Resposta final

Boas Práticas — Desenvolvimento com IA

Com IA / Agentes (Claude & Gemini)

- **Evite o "Vibe Coding":** Não peça para a IA codar logo de cara. Exija o fluxo: Explore (leia arquivos) → Planeje (proponha solução) → Implemente.
- **O Padrão "Documentar e Limpar":** Sessões longas fazem a IA alucinar. Peça para a IA resumir o progresso num arquivo `.md`, use `/clear` (zere a memória) e retome a partir do documento.
- **Revisão Cruzada (Multi-Agente):** A mesma sessão que escreveu o código é ruim para revisá-lo. Abra um terminal fresco para revisar o PR de forma crítica.
- **CLAUDE.md:** Mantenha-o enxuto (max 200 linhas). Documente nele os erros que a IA costuma repetir no seu projeto.

Com Snowflake + Python

- Nunca armazene credenciais no código — use `.env`
- Adicione `.env` ao `.gitignore`
- Use queries parametrizadas (evite SQL injection)
- Feche conexões com `conn.close()` ou `with`
- Suspenda o warehouse após análises longas
- Prefira `pd.read_sql()` para resultados em DataFrame

> **Fluxo recomendado:** Prototipar no Jupyter → Revisar com Claude Code → Refatorar em script → Agendar no pipeline de dados da empresa.

Sua opinião importa

Responda a avaliação da Parte 2 do curso

PERGUNTA PRINCIPAL

"Qual seu grau de satisfação com a Parte 2 do curso?"

1 — Muito Insatisfeito ● ● ● ● ● 5 — Muito Satisfeito

forms.office.com/r/iPNssDMPR8

TECH THINK 2

Inteligência Artificial além da nuvem

Obrigado!

Foi um prazer construir isso com vocês.

INSTRUTOR

Vinicius Tertulino Parede

Engenheiro Eletricista

EMPRESA

Tech2Think

tech2think.com.br

CÓDIGO FONTE

`github.com/tech2think/python-learning`